

Number Systems

Lecture 2 – Intro to Info. & Comm. Technologies

Number system

*A **number system** defines how a number can be represented using distinct symbols.*

A number can be represented differently in different systems.

For example, the two numbers $(2A)_{16}$ and $(52)_8$ both refer to the same quantity, $(42)_{10}$, but their representations are different.

Data Representation in Computers

What is Data

Data is simply
any **numbers**, **letters** or **symbols** that can
be entered into a computer system.

Data values **don't have any**
meaning unless we put them into **context**

Information = **Data** + **Context**

Computers Process Data

- Computers are used to process all types of information in a broad spectrum fields.
 - **Numeric data** consisting of Integers and real numbers are used in programs calculating payroll. We typically perform **arithmetic operations** on numeric data.
 - **Strings** of **alphabets** and numbers (**Alphanumeric Data**) are processed in customer record keeping systems.
 - **Multimedia** content including images, sound and text are frequently used in a large collection of application areas.
 - **Signals** representing various types of information like temperature, pressure, presence or absence of objects etc. are processed by computers in Robotics, IoT, monitoring and control applications.

Numbers

❑ Natural Numbers:

- ❑ A natural number is a number that comes naturally,
- ❑ Natural numbers are greater than zero we can use this numbers as counting numbers: $\{1, 2, 3, 4, 5, 6, \dots\}$.

❑ Integers:

- ❑ Integers include the Natural numbers, zero, and the negative Natural numbers.
- ❑ Numbers in the form of negative and positive numbers $\{\dots, -4, -3, -2, -1, 0, 1, 2, 3, 4, \dots\}$.

How is Data Actually Stored in Computer

Everything that is stored and processed inside a computer *(all data, information, instructions, files, images, etc.)* is stored as **Binary Numbers**

Digital computers have been made such that all data and instructions(program) for processing must be stored in computers memory before processing.

How to store *text* and *pictures* as numbers?

- The solution is to use **numeric codes**:
 - Different **letters** in a text document are given different numeric codes
 - Different **pixels** (colored dots) in an image are given different numeric codes
 - Different **sounds** in a music file are given different numeric codes

Everything is numbers!

Memory Measuring Units

(As viewed by computer scientists)

- Bits can be grouped together to make them easier to work with. A group of 8 bits is called a **byte**.

Most computers can process millions of bits every second. A hard drive's storage capacity is measured in gigabytes or terabytes. RAM is often measured in megabytes or gigabytes.

Bt

1. bit = From a shortening of the words “**b**inary **dig**it”
1. the basic unit of information for computers
1. 1 or 0 are the only values

Byte

1. Byte = 8 bits

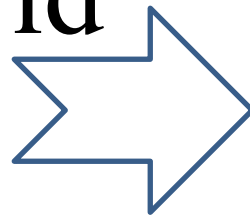
1. Each keyboard character is stored as a Byte

3. Example A = 01000001

This is an excellent
diagram

Text to binary conversion

hello world



01101000 01100101
01101100 01101100
01101111 00100000
01110111 01101111
01110010 01101100
01100100

sea = (011100110110010101100001)₂

How Images are Stored in Memory

0	0	255	0	0
0	0	255	0	0
0	0	255	0	0
255	255	255	255	255
0	0	255	0	0
0	0	255	0	0
0	0	255	0	0

How To Represent Color Images in Memory

RGB

Three Separate Matrices for R, G and B, respectively.

How To Represent Color Images in Memory

How can we represent a character?

- IDEA.
 - Assign numeric codes to characters and represent each character in a Byte using it's numeric code.
 - Can we assign numeric codes of our choice to each character?. What might be a problem with this approach?

How can we represent a character?

- **IDEA**
- Create a Standard coding scheme so that information can be easily shared between devices from different vendors.
- Standard Codes
 - ASCII (American Standard Code for Information Interchange)
 - Unicode
 - Unicode Transformation Format(UTF) UTF-8, UTF-16
 - ANSI Character Set

ASCII Character Encoding

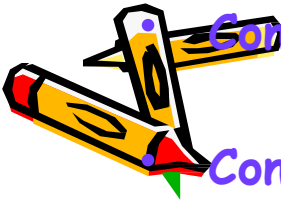
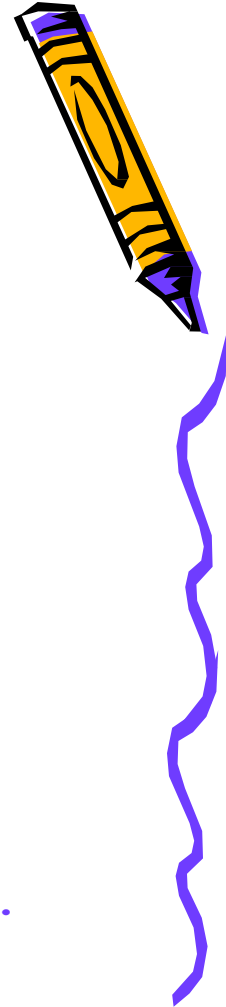
ASCII to Binary 0s and 1s

Number Systems

Number System Name	Number System and Description
Decimal Number System	Base 10 Digits used: 0 to 9
Binary Number System	Base 2 Digits used : 0, 1
Octal Number System	Base 8. Digits used : 0 to 7
Hexadecimal Number System	Base 16. Digits used: 0 to 9, Letters used : A- F

Conversion Types

- Conversion between Binary & Decimal.
- Conversion between Decimal & Hexadecimal.
- Conversion between Hexadecimal & Binary.
- Conversion between Decimal & Octal.
- Conversion between Octal & Hexadecimal.
- Conversion between Binary & Octal.



Binary Number System

- Each digit of a binary number is called a bit. For example the number $(1010)_2$ has 4 bits.
- The more the number of bits, the more the total numbers that can be represented. For example if we use 8 bits, then total numbers that can be represented by 8 bits is $2^8=256$.

Binary Number System (contd)

- Suppose we have 8 bits. We know that we can store 256 numbers in these 8 bits.
- If we store only positive numbers, then the smallest number that we can store is 0, and the largest number that we can store is $2^8-1= 255$.
- 0 in binary (using 8 bits) is $(00000000)_2$ and 255 in binary is $(11111111)_2$

How to compute the decimal value of a binary number

- Technique
 - Multiply each bit by 2^n , where n is the position of the digit. So, 2^n is the value of the bit.
 - The weight is the position of the bit, starting from 0 on the right
 - Add the results

How to compute the decimal value of a binary number

- **Example**
- Binary Number: 10101
- Calculating Decimal Equivalent:

Step	Binary Number	Decimal Number
Step 1	10101	$(1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$
Step 2	10101	$16 + 0 + 4 + 0 + 1$
Step 3	10101	21

How to compute the decimal value of a binary number (shortcut)

Suppose we want to find the decimal value of binary number 10101011.

128	64	32	16	8	4	2	1
1	0	1	0	1	0	1	1

Simply add the decimal values where the corresponding bit is 1.

Hexadecimal Number System

The **hexadecimal number system** has 16 digits. 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Examples of hexadecimal numbers:

1. 1F
2. 1AB
3. 100
4. 54

How to compute the decimal value of a hexadecimal number

- Technique
 - Multiply each digit by 16^n , where n is the position of the digit. So, 16^n is the value of the digit.
 - positions of the bit start from 0 on the right
 - Add the results

How to compute the decimal value of a hexadecimal number

Step	Hex Number	Decimal Number
Step 1	19FDE	$(1 \times 16^4) + (9 \times 16^3) + (F \times 16^2) + (D \times 16^1) + (E \times 16^0)$
Step 2	19FDE	$(1 \times 16^4) + (9 \times 16^3) + (15 \times 16^2) + (13 \times 16^1) + (14 \times 16^0)$
Step 3	19FDE	$65536 + 36864 + 3840 + 208 + 14$
Step 4	19FDE	106462

Decimal Number Systems

- One of the most common number systems.
- Has 10 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Decimal to Binary

- Technique
 - Divide by two, keep track of the remainder

$$125_{10} = ?_2$$

2		125	
		62	1
2		31	0
		15	1
2		7	1
		3	1
2		1	1

$$125_{10} = 1111101_2$$

Decimal to Hexadecimal

$$1234_{10} = ?_{16}$$

$$\begin{array}{r} 16 \overline{) 1234} \\ 16 \overline{) 77} \\ 16 \overline{) 7}4 \\ 0 \end{array}$$

$$\begin{array}{l} 2 \\ 13 = D \\ 4 \end{array}$$

$$1234_{10} = 4D2_{16}$$

Converting Hexadecimal to Binary and Vice Versa

Hexadecimal to Binary

- We know that the largest digit in hexadecimal is F, and this digit requires 4 bits for binary representation.
- So, find 4-digit binary value of each hexadecimal digit.
- The binary equivalent of the hexadecimal number is simply the concatenation of 4-digit binary values of all digits in order.

Conversion Chart

Decimal	Binary	Hex
00	0000	0
01	0001	1
02	0010	2
03	0011	3
04	0100	4
05	0101	5
06	0110	6
07	0111	7
08	1000	8
09	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Hexadecimal to Binary

Example

$$10AF_{16} = ?_2$$

1	0	A	F
↓	↓	↓	↓
0001	0000	1010	1111

$$10AF_{16} = 0001000010101111_2$$

Binary to Hexadecimal

- Make Quadruplets of bits from right to left.
- Find the hexadecimal value of each quadruplet.
- The final hexadecimal number is the concatenation of hex digits in order.

Binary to Hexadecimal

Example:

Octal to Decimal

✓ **Technique**

Multiply each bit by 8^n , where n is the “weight” of the bit.

The weight is the position of the bit, starting from 0 on the right.

Add the results

Example

$$\begin{array}{rcll} 724_8 & \Rightarrow & 4 \times 8^0 & = 4 \\ & & 2 \times 8^1 & = 16 \\ & & 7 \times 8^2 & = 448 \\ & & & \text{---} \\ & & & 468 \end{array}$$

10

Decimal to Octal

Technique

Divide by 8

Keep track of the remainder

Example

Octal To Binary

Binary to Octal

- Technique
 - Group bits in threes, starting on right
 - Convert to octal digits

$$\text{— } 1011010111_2 = ?_8$$

Octal to Hexadecimal

Technique

Use binary as an intermediary

$$1076_8 = ?_{16}$$

Octal to Hexadecimal

Technique

*Use binary as an
intermediary*

$$1 \quad AF_{16} = ?_8$$

Representing Signed Binary Numbers in Computer Memory

Signed Magnitude Representation

- Signed binary numbers can be represented in computer memory by using the most significant bit as a signed bit. This representation is also called sign & magnitude representation.
- If the signed bit is on, the number is negative, and if the signed bit is off, the number is positive.
- **What is the total number of values that can be stored using this signed representation?**

Signed Magnitude Representation

Examples:

- $(01101101)_2 = +(109)_{10}$
- $(11101101)_2 = -(109)_{10}$
- $(00101011)_2 = +(43)_{10}$
- $(10101011)_2 = -(43)_{10}$

Signed Magnitude Representation

- Range of values for positive number:
1 to 127
- Range of values for negative numbers:
-127 to -1.

Problem:

- Positive zero and negative zero.
- So, The range for positive numbers becomes +0 to +127, and for negative numbers the range becomes -127 to -0.

Converting a Binary Number Represented by Signed Magnitude into Decimal.

- Suppose there are N bits in the given binary number represented by signed magnitude form. The most significant bit will be used to represent the sign.
- Simply find the decimal equivalent of remaining $N-1$ bits, and put $-$ sign if the sign bit is 1 else put $+$.
- **Example:** (11010001)
Find the decimal of (1010001) which is 81. Since the sign bit is 1, so the number is -81.

1's Complement

- To represent a negative number in 1's complement, first find its positive binary counterpart.
- Then inverse the bits.
- **For example:** Represent -2 using 1 byte in 1's complement.
 - 2 = 00000010
 - -2 = 11111101

Note: Applying 1's complement again on the binary of a negative number (represented by 1's complement) will convert it into the positive number.

Range of 1's Complement Representation

- The range of signed numbers using one's complement is $-(2^{N-1} - 1)$ to $(2^{N-1} - 1)$ with ± 0 .
- **For example:** if we use 8 bits, then range is -127_{10} to $+127_{10}$ with zero being either 00000000 (+0) or 11111111 (-0).

Converting a Binary Number Represented by 1's Complement into Decimal.

- Suppose there are N bits in the given binary number represented by 1's complement.
- If the most significant bit is 0, then the number is positive, and simply find its decimal equivalent.
- If the most significant bit is 1, then the number is negative. So, first apply 1's complement (to convert the negative binary number into positive binary number), then find the decimal equivalent and put – sign.
- **Example:** (11010011)
 1. Finding 1's complement since msb is 1: 00101100.
 2. Now find the decimal equivalent: 44
 3. Put negative sign. (-44)

2's complement

- First find 1's complement.
- Now add 1 to the 1's complement.
- It does not have positive and negative zero problem.
- **For example:** Represent -2 using 1 byte in 2's complement.
 - ❑ $2 = 00000010$
 - ❑ 1's complement = 11111101
 - ❑ 2's complement = $11111101 + 1 = 11111110$
 - ❑ $-2 = 11111110$

Note:

Applying 2's complement again on the binary of a negative number (represented by 2's complement) will convert it into the binary of the same positive number. For example, apply 2's complement on the binary of -2 (calculated above) and see the result.

Range of 2's Complement Representation

- The range of signed numbers using two's complement is $-(2^{N-1})$ to $(2^{N-1} - 1)$. There is only one representation of 0 in this representation.
- **For example:** if we use 8 bits, then range is -128_{10} to $+127_{10}$ with zero being 00000000.

Converting a Binary Number Represented by 2's Complement into Decimal.

- Suppose there are N bits in the given binary number represented by 2's complement.
- If the most significant bit is 0, then number is positive and simply find its decimal equivalent.
- If the most significant bit is 1, then number is negative. So, first apply 2's complement (to convert it into positive binary number), then find the decimal equivalent of positive binary number and put – sign.
- **Example:** (11010011)
 1. Finding 2's complement since msb is 1: 00101101.
 2. Now find the decimal equivalent: 45
 3. Put negative sign. (-45)

Recommended

- <https://www.youtube.com/watch?v=1GSjbWt0c9M>
- <https://www.khanacademy.org/computing/computer-science/how-computers-work2/v/khan-academy-and-codeorg-introducing-how-computers-work>
- <https://www.youtube.com/watch?v=ptzGI9VaZmQ>